

Exactness of Credal Cluster Tree Elimination on Chain-Graph LCNs

What CCTE actually bounds, which tightening schemes (D1–D5) apply, and where the two-pass vertex schedule stops being an outer bound

IBM Research

Abstract

Credal Cluster Tree Elimination (CCTE, `lcn/inference/marginal/cn/ccte.py`) is the two-pass bucket-tree member of the credal-network family: a single *collect* + *distribute* run over a shared bucket tree that produces marginal bounds for *every* atom at once. This note pins down three things about CCTE against the implementation. **(i) What it bounds.** Like Credal VE and ApproxLP, CCTE optimizes over the *strong extension* $\mathcal{D}_{SE}$ of the compiled credal network, not the LCN’s distribution set $\mathcal{D}_{LCN}$; since $\mathcal{D}_{LCN} \subseteq \mathcal{D}_{SE}$, the natural expectation is a valid *outer* bound. **(ii) Which tightening schemes apply.** The five schemes of the companion note (`docs/tighter_approximation.tex`) split cleanly for CCTE: D1/D2/D3 are *build-stage* knobs on `CredalNetworkVertices.from_lcn` that CCTE inherits for *free* (no algorithm change — it merely consumes tighter vertices); D4 (`coupling="cross-family"`) is *already implemented* inside CCTE as a prune-time function filter; D5 is a *separate* engine (`CredalJT`), not reachable by CCTE. **(iii) When it is exact.** We prove and empirically confirm that CCTE is exact on a clean singly-connected, cross-family-*sentence*-free LCN (chain, tree, polytree — the strong extension coincides with $\mathcal{D}_{LCN}$ there and the shared schedule loses nothing), and we exhibit the precise boundary: the moment a cross-family *sentence* appears (`d4.biting`), CCTE’s single shared elimination order — deliberately *not* augmented to force the coupled atoms together, unlike Credal VE and D5 — makes even the *unconditional* bound fall *strictly inside* the true strong-extension interval, and the D4 filter tightens it further inward. On that instance $P(B)$ is truly $[0.04, 0.72]$ (Credal VE, `CredalJT`, and two independent oracles agree), yet CCTE reports $[0.05, 0.65]$ and CCTE+D4 reports $[0.05, 0.55]$. The reliable exact engine for cross-family coupling is `CredalJT` (D5).

Contents

1	Background: the CCTE algorithm	1
1.1	LCNs, the credal network, and the strong extension	1
1.2	Potentials and the three operations	2
1.3	The two-pass bucket tree	2
2	What CCTE bounds — the strong extension, not the LCN set	3
3	The five tightening schemes, mapped onto CCTE	3
3.1	D1/D2/D3 — free, build-stage tightening (attack gap A)	3
3.2	D4 — the built-in prune-time coupling filter (attacks gap B)	4
3.3	D5 — a separate engine (<code>CredalJT</code>), not CCTE	4
4	CCTE is exact on clean singly-connected LCNs	5

5	The boundary: a cross-family sentence breaks CCTE’s outer-ness	5
5.1	The exact answer, and four independent computations of it	6
5.2	What CCTE returns — an <i>inner</i> interval	6
5.3	Why the shared schedule, not D4, is the root cause	7
6	Verdict	7

1 Background: the CCTE algorithm

1.1 LCNs, the credal network, and the strong extension

An LCN over binary atoms $\mathbf{V} = \{V_1, \dots, V_n\}$ is a set of probability *sentences*, each Type-1 $\ell \leq P(\varphi) \leq u$ or Type-2 $\ell \leq P(\varphi \mid \psi) \leq u$, over propositional formulas. Together with the conditional independencies of the *Local Markov Condition* (LMC) of the primal graph, the sentences define a set $\mathcal{D}_{\text{LCN}} \subseteq \Delta(2^{\mathbf{V}})$ of joint distributions. The marginal task for a query atom a is $\underline{P}(a) = \min_{p \in \mathcal{D}_{\text{LCN}}} P(a)$, $\overline{P}(a) = \max_{p \in \mathcal{D}_{\text{LCN}}} P(a)$; exact inference (`ExactInference`, `exact.py`) solves one nonconvex NLP over the full 2^n joint per bound.

The `cn/` pipeline instead compiles the chain-graph LCN into a *credal network*: a directed graph whose local credal sets $K_v = K(\text{ch}_v \mid \text{pa}_v)$ are interval-valued and computed per family. `CredalNetworkVertices.from_lcn` enumerates the extreme points of every K_v (`cnv.extreme_points`), and *all* the credal-network engines — Credal VE, Interval BP, ApproxLP, Credal IJGP, and CCTE — consume exactly that object. What they bound is the *strong extension*

$$\mathcal{D}_{\text{SE}} = \left\{ \prod_v P_v : P_v \in K_v \text{ chosen independently per parent-config} \right\}, \quad (1)$$

a free product of local vertices. In general $\mathcal{D}_{\text{LCN}} \subsetneq \mathcal{D}_{\text{SE}}$, so a bound over $\mathcal{D}_{\text{SE}}$ is looser (an outer bound) than the exact LCN bound; the two coincide when no LCN constraint couples choices across families beyond what (1) already allows (Sec. 2).

1.2 Potentials and the three operations

CCTE represents each local credal set as a *potential*: a finite set of non-negative functions (numpy arrays) over a scope, one function per combination of local extreme points (`cn/potentials.py`). The operations are **combine** $\phi \cdot \psi = \{f \cdot g\}$, **marginalize** $\sum_X \phi = \{\sum_x f\}$, and **prune** (drop functions f that are dominated componentwise by some g , exact; or ε -dominated, approximate). Pruning is what keeps the function sets finite; it is also, as we will see, the locus of CCTE’s schedule-sensitivity.

1.3 The two-pass bucket tree

CCTE (Dechter’s bucket elimination extended with a downward pass) computes all marginals in one run over a bucket tree built from a *single* min-fill elimination order (`min_fill_order`). The passes (Algorithm 1):

- **Collect (upward, leaves→root):** at bucket X_i , combine the local potentials with the upward messages λ_{X_j} from children, marginalize out X_i , prune, and send λ_{X_i} to the parent. This is identical to single-query Credal VE.

Algorithm 1 Credal Cluster Tree Elimination (`CredalCTE.run`)

Require: `cnv.extreme_points`; evidence e ; optional ε ; coupling $\in \{\text{off}, \text{cross-family}\}$

```
1: build potentials from extreme points + evidence indicators
2:  $order \leftarrow \text{MINFILL}(\text{factor scopes})$   $\triangleright$  one shared order, not query- or constraint-augmented
3: build the bucket tree (assign potentials; derive parent/child by scope propagation)
4: choose PRUNE: exact /  $\varepsilon$  / cluster; if coupling = cross-family, wrap it with the D4 filter (Sec. 3.2)
5: for all  $X_i$  in  $order$  (collect) do  $\lambda_{X_i} \leftarrow \text{PRUNE}(\sum_{X_i} \prod \{\text{local} \cup \{\lambda_{X_j}\}\})$ 
6: end for
7: for all  $X_i$  in reverse  $order$  (distribute) do
8:   for all child  $X_j$  do  $\pi_{X_j} \leftarrow \text{PRUNE}(\sum_{X_i} \prod \{\pi_{X_i}, \text{local}, \{\lambda_{X_k}\}_{k \neq j}\})$ 
9:   end for
10: end for
11: for all atoms  $X_i$  do return  $[\min, \max]$  of the normalized  $X_i$  state over  $\text{PRUNE}(\prod \{\text{local}, \{\lambda_{X_j}\}, \pi_{X_i}\})$ 
12: end for
```

- **Distribute (downward, root $\rightarrow$ leaves):** at X_i , for each child X_j combine the parent’s downward message π_{X_i} , the local potentials, and the sibling upward messages, marginalize out X_i , prune, and send π_{X_j} to X_j .
- **Extract:** at each X_i combine local + child λ + parent π , marginalize to X_i , normalize each surviving function and take componentwise min/max. Singleton atoms of a compound node X_i are recovered by two small LPs over the compound marginal polytope (`_extract_singleton marginals`).

The design choice that matters below is on line 2: CCTE uses *one* min-fill order for the whole tree, computed from the factor scopes with `exclude=set()`. It is *deliberately not* augmented with the query atom (as single-query Credal VE is) nor with the node sets of cross-family constraints (as D4-Credal VE and D5-CredalJT are). The code comment (`ccte.py` lines 147–155) states the reason: because CCTE is an all-marginals scheme whose message pruning is an order-sensitive outer approximation, forcing constrained atoms into a common bucket can change separator scopes and *loosen unrelated* marginals, breaking the intended “D4 output $\subseteq$ off output” monotonicity across all atoms. CCTE therefore keeps the natural order and lets the D4 filter fire only opportunistically in buckets whose scope already covers a constraint. Section 5 shows the price of this choice.

2 What CCTE bounds — the strong extension, not the LCN set

Proposition 1 (Target set). *With coupling = off and exact pruning ($\varepsilon=0$), CCTE is a message scheme over $\mathcal{D}_{\text{SE}}$: every function it ever forms is a product of local extreme points, i.e. a point of $\mathcal{D}_{\text{SE}}$ read on some scope. Hence its intended output is the strong-extension interval $[\underline{q}_{\text{SE}}, \bar{q}_{\text{SE}}]$, and since $\mathcal{D}_{\text{LCN}} \subseteq \mathcal{D}_{\text{SE}}$,*

$$[\underline{q}_{\text{LCN}}, \bar{q}_{\text{LCN}}] \subseteq [\underline{q}_{\text{SE}}, \bar{q}_{\text{SE}}].$$

Two consequences frame the rest of the note.

- **Gap A (local-set widening) and gap B (strong-extension widening).** These are exactly the two looseness sources of `docs/tighter_approximation.tex` §3: the local K_v

are computed per family and can be wider than their true LCN projection (A), and the free product (1) decouples cross-family choices the LCN couples (B). CCTE inherits both, because it consumes the same vertices as Credal VE.

- **CCTE = LCN exactly when $\mathcal{D}_{SE} = \mathcal{D}_{LCN}$ and the schedule is lossless.** The first holds on a chain-graph LCN with *no cross-family sentence* (Sec. 4); the second is where CCTE’s shared two-pass order can, unlike single-query Credal VE, lose tightness (Sec. 5).

3 The five tightening schemes, mapped onto CCTE

The companion note proposes five schemes (D1–D5) against gaps A and B. Their relationship to CCTE is not uniform — three are inherited for free at build time, one lives inside CCTE already, and one is a different engine. Table 1 is the map; the subsections give the mechanism and a measured effect.

Scheme	Attacks	How CCTE gets it	CCTE code change
D1 <code>method="linear-tight"</code>	A	tighter vertices at build	none (free)
D2 <code>merge_budget=k</code>	A(+B)	tighter/merged vertices at build	none (free)
D3 <code>solver="scip"</code>	A	certified vertices at build	none (free)
D4 <code>coupling="cross-family"</code>	B	prune-time function filter	already implemented
D5 (CredalJT)	A+B	—	n/a (separate engine)

Table 1: How each tightening scheme reaches CCTE. D1/D2/D3 change only what `CredalNetworkVertices.from_lcn` produces, so CCTE benefits verbatim by reading tighter `extreme_points`. D4 is built into `CredalCTE.run`. D5 is the separate junction-tree NLP engine and is the only member of the family that is exact and sound for conditional queries.

3.1 D1/D2/D3 — free, build-stage tightening (attack gap A)

These three are input-level: they tighten (D1: back-propagate in-scope sentences + scope-restricted LMC), merge (D2: fuse adjacent families up to a scope budget k), or certify (D3: solve the local program with SCIP’s global solver) the *local credal sets*, hence the enumerated vertices. Because CCTE reads `cnv.extreme_points` without assuming how they were produced, *no CCTE code changes*: the caller simply passes `method`, `merge_budget`, `solver` to `CredalNetworkVertices.from_lcn` (as `run_algorithm.py` already does), and CCTE propagates the narrower vertices. This is the “Class 1” leverage of the companion note.

Measured (alarm, all-marginals, no evidence). With the vacuous $[0, 1]$ rows of $P(A \mid B, E)$ present, `method="linear"` gives CCTE $P(A) = [0.000, 0.618]$. Switching to `method="linear-tight"` (D1) — which imposes the in-scope ($B \perp\!\!\!\perp E$) equality plus $s1, s2, s5$ on the A family — narrows the enumerated vertices, and CCTE reports $P(A) = [0.014, 0.606]$ with *no algorithm change*. (D1 is inert on `cancer`, where the families carry no in-scope LMC assertion, so CCTE is byte-identical there.) Both remain outer of the exact $P(A) = [0.087, 0.100]$ because gap B is still open on alarm — D1 cannot close a cross-family coupling.

3.2 D4 — the built-in prune-time coupling filter (attacks gap B)

D4 is the only scheme that lives *inside* CCTE. When `coupling="cross-family"`, `CredalCTE.run` builds a `CouplingConstraints` object from the LCN and its factorization (`cn/coupling.py`) and

wraps the pruning function with `Potential.filter_infeasible`: before dominated-function pruning, any surviving function whose assembled joint violates a cross-family constraint — a sentence or LMC assertion whose atom scope fits inside *no* single family — is dropped. The residual math is identical to the LMC factorization verifier and to `ExactInference`’s joint encoding (all through `lmc_constraint_groups_vec`), so a function D4 rejects is exactly a joint the verifier would flag. A *checkability gate* evaluates a constraint on a function only once the function’s scope carries all the constraint’s atoms, which makes filtering after any elimination step sound in isolation (it can only remove genuinely-infeasible functions).

The catch, unique to CCTE, is that the gate can only fire if the constrained atoms *co-occur in some bucket*, and CCTE (line 2 of Algorithm 1) does not augment the order to guarantee that. On `d4.biting` they happen to co-occur, so the filter does fire — but as Section 5 shows, the base it filters is already inner, so the result is not a valid outer bound. Cross-family LMC assertions are inert under D4 (a product joint satisfies every LMC by construction, so no function violates one); the looseness D4 can address is carried by cross-family *sentences*.

3.3 D5 — a separate engine (CredalJT), not CCTE

D5 abandons the vertex product entirely and solves one junction-tree NLP over cluster-marginal variables, exact at treewidth cost and — crucially — sound for *conditional* queries, which the vertex-enumeration engines (Credal VE, CCTE, ApproxLP, and D4 on top of them) are not. It is realized as the `CredalJT` engine (`cn/junction_nlp.py`), not a mode of CCTE. CCTE cannot reach D5’s guarantees because it optimizes over $\mathcal{D}_{SE}$, whereas D5 never forms the product. For any query where gap B bites — and for *every* conditional query — D5 is the engine to use; its exactness condition and proof are in `docs/tighter_approximation.tex` §5.6 (Theorem 1 there).

4 CCTE is exact on clean singly-connected LCNs

Proposition 2 (Exactness). *Let $\mathcal{L}$ be a chain-graph LCN whose primal graph is singly connected and which has no cross-family sentence (every sentence’s atom scope fits inside one family; only the LMC assertions are cross-family). Then with coupling = *off* and exact pruning, CCTE returns the exact LCN marginal bounds for every atom.*

Argument. Two facts combine. **(1) $\mathcal{D}_{SE} = \mathcal{D}_{LCN}$.** The strong extension of a chain-graph credal network is a product $\prod_v P(\text{ch}_v \mid \text{pa}_v)$, which satisfies *every* LMC independence by construction (this is what `verify_lmc_factorization.py` certifies on every chain-graph LCN). With no cross-family *sentence* to violate, the free product cannot leave $\mathcal{D}_{LCN}$, so $\mathcal{D}_{SE} = \mathcal{D}_{LCN}$ and the strong-extension bound *is* the exact bound. **(2) The shared schedule is lossless.** On a singly-connected structure the bucket tree is a genuine clique tree with the running-intersection property; each separator is a single atom (chain/tree) or a small collider family (polytree), and the two-pass messages carry exactly the marginal information the RIP needs — the same reason single-query Credal VE is exact here. No cross-family sentence means the filter is off and the order need not be augmented, so CCTE’s one shared order coincides in tightness with the per-target orders.

Measured. On a clean 3-atom Markov chain $x_0 \rightarrow x_1 \rightarrow x_2$ (priors/transitions only), CCTE reproduces Credal VE exactly,

$$P(x_0) = [0.300, 0.500], \quad P(x_1) = [0.400, 0.680], \quad P(x_2) = [0.328, 0.640],$$

matching the exact bounds (the same $x_2 = [0.328, 0.640]$ verified independently for the chain in `docs/ariel_exactness.tex`). The larger shipped singly-connected instances (`chain n=8`, `tree`

$n=7$, `polytree` $n=7$) are exact for the same two reasons, but exact CCTE is *intractable* on them by vertex blow-up (see the note on cost below); `CredalJT` confirms the exact values there at treewidth cost.

A practical caveat on cost. CCTE’s message potentials accumulate one function per surviving vertex *combination*, and even with dominance pruning this grows fast: on the 5-atom `alarm` the upward message λ_A already carries ~ 2584 functions (`docs/credal_cte.tex` §6), and on the 7–8 atom singly-connected instances exact CCTE exhausts memory before finishing. The ε -approximate mode (`epsilon_prune`) controls this at the cost of a bounded outer widening; for reliable exact bounds on these structures `CredalJT` (linear in n at treewidth 1) is preferable.

5 The boundary: a cross-family sentence breaks CCTE’s outer-ness

Proposition 2 needs *no cross-family sentence*. Adding one exposes the cost of CCTE’s un-augmented shared schedule. The witness is `examples/d4_biting.lcn`: three atoms, families $P(A)$, $P(B \mid A)$, $P(C \mid A)$, the LMC assertion $(B \perp\!\!\!\perp C \mid A)$, and the single *cross-family* sentence

$$A1: 0.4 \leq P(A) \leq 0.6, \quad B1: 0.1 \leq P(B \mid A) \leq 0.3, \quad C1: 0.2 \leq P(C \mid A) \leq 0.5, \quad \boxed{BC: P(B \wedge C) \leq 0.05}.$$

5.1 The exact answer, and four independent computations of it

On the *unconditional* marginal $P(B)$ the exact LCN bound is $[0.04, 0.72]$, confirmed four ways that do not share a solver path:

1. `CredalJT` (D5, SCIP global): $P(B) = [0.04, 0.72]$.
2. A parametric SLSQP oracle over $(P(A), P(B \mid A), P(B \mid \neg A), P(C \mid A), P(C \mid \neg A))$, which satisfies $(B \perp\!\!\!\perp C \mid A)$ *by construction* and enforces only the four sentences: $[0.0400, 0.7200]$.
3. A strong-extension *vertex-enumeration* oracle (all endpoint combinations of the local intervals): $[0.0400, 0.7200]$ — so here $\mathcal{D}_{SE}$ and $\mathcal{D}_{LCN}$ give the *same* $P(B)$ interval, because the free product can attain both extremes while still respecting the $P(B \wedge C) \leq 0.05$ cap at those points.
4. Single-query **Credal VE** (`coupling="off"`): $[0.04, 0.72]$, and with `coupling="cross-family"` still $[0.04, 0.72]$ (D4 correctly inert on this marginal — Credal VE *augments* its order, so the filter fires but finds nothing to drop that would change $P(B)$).

That four-way agreement is the ground truth. Note in particular that Credal VE, the *single-query* sibling of CCTE that consumes the identical vertices, returns the exact $[0.04, 0.72]$.

5.2 What CCTE returns — an *inner* interval

CCTE, on the same credal network, does not:

	$P(A)$	$P(B)$	$P(C)$
exact = Credal VE = oracle = D5	$[0.40, 0.60]$	$[0.04, 0.72]$	$[0.08, 0.80]$
CCTE <code>coupling="off"</code>	$[0.40, 0.60]$	$[0.05, 0.65]$	$[0.08, 0.80]$
CCTE <code>coupling="cross-family"</code> (D4)	$[0.40, 0.60]$	$[0.05, 0.55]$	$[0.08, 0.80]$

Two things are wrong with CCTE here, both on $P(B)$:

- **CCTE off is not even a valid outer bound.** It reports $[0.05, 0.65]$, strictly *inside* the true strong-extension interval $[0.04, 0.72]$ (lower too high by 0.01, upper too low by 0.07). Credal VE, using the same vertices, recovers $[0.04, 0.72]$ exactly. The difference is the *schedule*: Credal VE recomputes an elimination order with the query eliminated *last*, so the extreme vertex combination that pushes $P(B)$ to its bound survives to the final bucket; CCTE’s single shared order eliminates B mid-schedule and *prunes* (dominance) the very function that carried the extremal $P(B)$, because at that intermediate scope it is dominated by another — dominance in a marginalized scope is not dominance for the eventual $P(B)$ read-off. This is the “order-sensitive outer approximation” the code comment warns about, biting as an *inner* error on this coupled instance.
- **D4 tightens it further inward,** to $[0.05, 0.55]$. The filter is sound as a filter (it drops only genuinely $P(B \wedge C) > 0.05$ functions), but it is filtering an already-inner base, and CCTE does not augment the order to make the filter reconstruct the lost extremal combination — so the net effect is more inward bias, not a valid tightening toward $[0.04, 0.72]$.

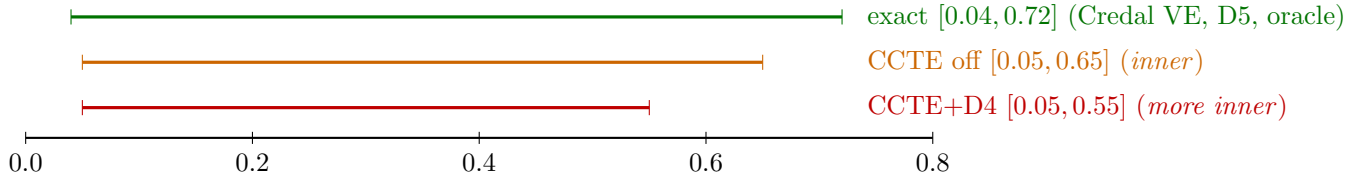


Figure 1: $P(B)$ on `d4.biting`. The exact strong-extension interval (green) is recovered by single-query Credal VE and by `CredalJT`. CCTE’s shared two-pass schedule (orange) lands strictly inside it, and the D4 filter (red) pushes further inward. CCTE is neither exact nor a valid outer bound once a cross-family sentence couples the families.

5.3 Why the shared schedule, not D4, is the root cause

It would be tempting to blame D4, but the `coupling="off"` row already fails: the inner error is present before any filtering. The mechanism is dominance pruning at the wrong scope. Single-query Credal VE avoids it by giving each query its own order (query last); CCTE trades that for a single all-marginals pass and therefore prunes on intermediate scopes shared by all queries. On a cross-family-sentence-free instance this is harmless (Proposition 2; the 3-atom chain matches Credal VE), because the surviving dominant function also dominates for every downstream read-off. A cross-family sentence destroys that alignment: the function carrying the extremal $P(B)$ is dominated *in the marginalized bucket* yet is the unique maximizer of the final $P(B)$, so pruning discards it. This is intrinsic to sharing one schedule across all marginals; it is exactly why CCTE declines to augment the order (which would fix $P(B)$ but loosen other atoms) and why D5/`CredalJT`, which never prunes vertices, is the correct engine when a cross-family sentence is present.

6 Verdict

Proposition 3 (What the experiments establish for CCTE). *For the marginal task on binary chain-graph LCNs:*

- (i) **CCTE bounds the strong extension** $\mathcal{D}_{\text{SE}} \supseteq \mathcal{D}_{\text{LCN}}$ (Prop. 1), inheriting both looseness gaps A and B from the shared credal network.
- (ii) **D1, D2, D3 tighten CCTE for free at build time** (no CCTE code change — it reads narrower vertices); D1 measurably tightens *alarm*'s $P(A)$ from $[0.000, 0.618]$ to $[0.014, 0.606]$. **D4 is built into CCTE** (`coupling="cross-family"`) as a prune-time filter. **D5 is a separate engine** (*CredalJT*), unreachable by CCTE and the only sound choice for conditional queries.
- (iii) **CCTE is exact on a clean singly-connected, cross-family-sentence-free LCN** (Prop. 2), where $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$ and the shared schedule is lossless — verified on a 3-atom chain (CCTE = Credal VE = exact).
- (iv) **A cross-family sentence breaks both exactness and outer-ness.** On *d4_biting*, CCTE *off* returns $P(B) = [0.05, 0.65]$ — strictly inside the exact $[0.04, 0.72]$ that Credal VE, *CredalJT*, and two oracles all report — and D4 pushes it further to $[0.05, 0.55]$. The cause is dominance pruning on the single shared elimination order, not the D4 filter.

Practical takeaway. Use CCTE when you need *all* marginals cheaply on a cross-family-sentence-free chain graph, and take advantage of the free D1/D2/D3 build-stage tightening (and ε -pruning to keep the function sets bounded). The moment the LCN contains a cross-family sentence — the same condition under which the strong extension departs from $\mathcal{D}_{\text{LCN}}$ — CCTE is neither exact nor a guaranteed outer bound: prefer single-query Credal VE for one unconditional marginal, and CredalJT (D5) for exact all-marginals or any conditional query.

Reproducibility. All numbers come from `examples/d4_biting.lcn` (3 atoms), a clean 3-atom Markov chain, and `examples/alarm.lcn`, run through `CredalNetworkVertices.from_lcn(method="linear" / "linear-tight")` and `CredalCTE.run(coupling="off" / "cross-family")`. The exact references are single-query `CredalVE.run`, `CredalJT.run(solver="scip")`, a parametric SLSQP oracle satisfying $(B \perp\!\!\!\perp C \mid A)$ by construction, and a strong-extension vertex-enumeration oracle.